

Sumário

1	Básico	1	5.1	Fórmulas	7
1.1	Código padrão (template)	1	5.2	MDC & MMC	8
1.2	Código padrão com casos teste (template)	1	5.3	Exponenciação Binária	8
2	Estruturas de Dados	1	5.4	Primos	8
2.1	PBDS (Set Indexado)	1	5.5	Crivo de Eratóstenes	8
2.2	BIT (Fenwick Tree)	1	5.6	Fatoração Prima	9
2.3	BIT 2D	2	5.7	Função Totiente de Euler (ϕ)	9
2.4	Union-Find (DSU)	2	6	Strings	9
2.5	Segment Tree	2	6.1	Arranjo de Sufixos (SA)	9
3	Grafos	2	6.2	KMP (Knuth-Morris-Pratt)	9
3.1	Ordenação Topológica	2	6.3	Rabin-Karp	10
3.2	Dijkstra	3	6.4	Trie	10
3.3	Bellman-Ford	3	6.5	Função Z	10
3.4	SPFA	4	6.6	Manacher	11
3.5	Floyd-Warshall	4	7	Árvores	11
3.6	Caminho Euleriano	4	7.1	LCA (Menor Ancestral Comum)	11
3.7	Componentes Biconexos (BCC)	5	7.2	Hash de árvore	11
3.8	Componentes Fortemente Conexas (SCC)	5	7.3	Decomposição Pesada-Leve (HLD)	11
3.9	Árvore Geradora Mínima (MST)	5	8	Geometria	11
4	Programação Dinâmica	6	9	Algoritmos de Ordenação	11
4.1	Edit Distance	6	9.1	Insertion Sort	11
4.2	Problema da Mochila	6	9.2	Merge Sort	11
4.3	Máscara de Bits	6	9.3	Quicksort	11
5	Matemática	7	9.4	Counting Sort	12
			10	Problemas Especiais	12

1 Básico

1.1 Código padrão (template)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

#define fastio ios::sync_with_stdio(0); cin.tie(0);
#define all(v) (v).begin(),(v).end()

int main() {
    fastio

    // code

    return 0;
}

/* Shell script
for a in {A..}; do
    cat base.cpp >> "$a.cpp"
done
*/
```

1.2 Código padrão com casos teste (template)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

#define fastio ios::sync_with_stdio(0); cin.tie(0);
#define all(v) (v).begin(),(v).end()

void solve() {
    // code
}

int main() {
    fastio

    int tc; cin >> tc;
    while (tc--) solve();

    return 0;
}
```

2 Estruturas de Dados

2.1 PBDS (Set Indexado)

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define ordered_set tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>
using namespace __gnu_pbds;

// ordered_set s;
// s.insert(10); s.insert(20); s.insert(30);
// O(LogN):
// s.find_by_order(1); // Retorna iterador para o 2º (s[i])
// s.order_of_key(30); // Retorna 2 (posição do valor X no set)
```

2.2 BIT (Fenwick Tree)

```
// O(LogN)
struct FenwickTree {
    vector<int> bit;
    int n;

    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    FenwickTree(vector<int> const& v) : FenwickTree(v.size()) {
        for (int i = 0; i < (int)v.size(); i++)
            add(i + 1, v[i]);
    }
}
```

```
// Soma prefixo [1, r]
int soma(int r) {
    int ret = 0;
    for (; r > 0; r -= r & (-r))
        ret += bit[r];
    return ret;
}

// Soma intervalo [l, r] (1-based)
int soma(int l, int r) {
    return soma(r) - soma(l - 1);
}

// Adiciona delta no índice idx (1-based)
void add(int idx, int delta) {
    for (; idx <= n; idx += idx & (-idx))
        bit[idx] += delta;
}
};
```

2.3 BIT 2D

```
// O(LogN * LogM)
struct FenwickTree2D {
    vector<vector<int>> bit;
    int n, m;

    FenwickTree2D(int n, int m) : n(n), m(m), bit(n + 1, vector<int>(m + 1, 0)) {}

    // Adiciona delta em (x, y) (1-based)
    void add(int x, int y, int delta) {
        for (int i = x; i <= n; i += i & (-i))
            for (int j = y; j <= m; j += j & (-j))
                bit[i][j] += delta;
    }

    // Soma prefixo [1,1] até (x, y)
    int soma(int x, int y) {
        int ret = 0;
        for (int i = x; i > 0; i -= i & (-i))
            for (int j = y; j > 0; j -= j & (-j))
                ret += bit[i][j];
        return ret;
    }

    // Soma retângulo (x1,y1) até (x2,y2) (1-based)
    int soma(int x1, int y1, int x2, int y2) {
        return soma(x2, y2)
            - soma(x1 - 1, y2)
            - soma(x2, y1 - 1)
            + soma(x1 - 1, y1 - 1);
    }
};
```

2.4 Union-Find (DSU)

```
// Oα(N) ≈ O(1)
vector<int> pai, tam;
void init(int n) {
    pai.resize(n+1);
    tam.assign(n+1, 1);
    iota(pai.begin(), pai.end(), 0); // pai[i] = i
}

int find(int x) {
    if (x == pai[x]) return x;
    return pai[x] = find(pai[x]); // path compression
}

// Union by Size: anexa a árvore menor na maior
void join(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return;
    if (tam[x] < tam[y]) swap(x, y);

    pai[y] = x;
    tam[x] += tam[y];
}
```

```
// Verifica se dois elementos estão no mesmo conjunto
bool mesmoConjunto(int x, int y) {
    return find(x) == find(y);
}

// Retorna o tamanho do componente que contém x
int tamanho(int x) {
    return tam[find(x)];
}
```

2.5 Segment Tree

```
struct SegTree {
    vector<ll> vet, tree;
    int n;

    SegTree(int n) : n(n) {
        tree.resize(4 * n);
        vet.resize(n + 1);
    }

    void build(int no, int l, int r) {
        if (l == r) {
            tree[no] = vet[l];
            return;
        }

        int m = 1 + (r - l) / 2;
        build(2 * no, l, m);
        build(2 * no + 1, m + 1, r);

        tree[no] = tree[2 * no] + tree[2 * no + 1];
    }

    void update(int no, int l, int r, int pos, ll val) {
        if (l == r) {
            tree[no] = val;
            return;
        }
        int m = 1 + (r - l) / 2;

        if (pos <= m) update(2 * no, l, m, pos, val);
        else update(2 * no + 1, m + 1, r, pos, val);

        tree[no] = tree[2 * no] + tree[2 * no + 1];
    }

    ll query(int no, int l, int r, int ql, int qr) {
        if (l > qr || r < ql) return 0;
        if (l >= ql && r <= qr) return tree[no];

        int m = 1 + (r - l) / 2;
        return m;
    }
};
```

3 Grafos

3.1 Ordenação Topológica

```
// O(V + E)
vector<vector<int>> graph;
vector<int> visited, processing, order;
bool hasCycle = false;

void dfs(int u, int p) {
    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (v == p) continue; // necessario em grafo nao
            -direcionado;
        if (processing[v]) {
            hasCycle = true;
            return;
        }
        if (!visited[v])
            dfs(v);
    }
}
```

```

    processing[u] = false;
    order.push_back(u);
}

// Detecta e imprime qlqr ciclo encontrado
vector<int> cycle;
bool buscarCiclo(int u) {
    if (processing[u] && cycle.empty()) {
        cycle.push_back(u);
        return cycle.size() == 1;
    }
    if (visited[u]) return false;

    visited[u] = true;
    processing[u] = true;

    for (auto& v : graph[u]) {
        if (buscarCiclo(v)) {
            cycle.push_back(u);
            return cycle.back() != cycle.front();
        }
    }

    processing[u] = false;

    return false;
}

int main() {
    for (int i = 1; i <= n; ++i)
        if (dfs(i)) break; // para de procurar ciclos
                             // se achar um

    if (cycle.size()) { // imprime ciclo achado
        cout << cycle.size() << '\n';
        reverse(cycle.begin(), cycle.end());

        for (auto& i : cycle)
            cout << i << ' ';
        cout << '\n';
    }

    return 0;
}

```

3.2 Dijkstra

```

//  $O((V + E)\log V)$  ou  $O(E \log V) - S(V)$ 
const ll N = 2e5 + 1, INF = 1e18;
vector<vector<pair<ll, ll>>> graph;
vector<ll> dist;
int n, m;

void dijkstra(int s) {
    dist.assign(n + 1, INF);
    priority_queue<pair<ll, ll>> pq;
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (-du > d[u]) continue; // otimização

        // v = vizinho, w = peso
        for (auto &[v, w] : graph[u]) {
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({-d[v], v}); // -d[v] p manter
                                     // ordenação crescente
            }
        }
    }
}

```

3.3 Bellman-Ford

```

// Calcula a menor distancia
// entre a e todos os vertices a
// detecta ciclo negativo

```

```

// Retorna 1 se ha ciclo negativo
// Nao precisa representar o grafo,
// soh armazenar as arestas
//
// Para achar ciclo positivo basta
// negar todos os pesos, logo
// um ciclo positivo vira negativo
//
// Para caminho máximo:
// Inicializar dist com -INF
// e funcao agora detecta ciclo
// positivo na n-ésima iteração
//
//  $O(V * E)$ 

const ll INF = 1e18; // 1e9 para int
struct Edge {
    int a, b, c;
};
vector<Edge> arestas;
vector<ll> dist, pai; // pai[i] = no anterior a i
int n, m;

bool bellman_ford(int s) {
    dist.assign(n + 1, INF);
    pai.assign(n + 1, -1);
    dist[s] = 0;

    int ultimo_relaxado;
    for (int i = 0; i < n; ++i) {
        ultimo_relaxado = -1;
        for (Edge& aresta : arestas) {
            if (dist[aresta.a] < INF && dist[aresta.b]
                > dist[aresta.a] + aresta.c) {
                dist[aresta.b] = dist[aresta.a] +
                    aresta.c;
                pai[aresta.b] = aresta.a;
                ultimo_relaxado = aresta.b;
            }
        }
    }

    return ultimo_relaxado != -1; // true se tem ciclo
                                   // negativo

// Retorna true se existe qualquer ciclo negativo
bool acharCicloNegativo() {
    dist.assign(n+1, 0);
    pai.assign(n+1, -1);

    ll ciclo;
    for (int i = 0; i < n; ++i) {
        ciclo = -1;
        for (Edge& a : arestas) {
            if (dist[a.a] < INF && dist[a.b] > dist[a.a]
                + a.c) {
                dist[a.b] = dist[a.a] + a.c;
                pai[a.b] = a.a;
                ciclo = a.b;
            }
        }
    }

    if (ciclo == -1) return false; // não tem ciclo
                                    // negativo

    for (int i = 0; i < n; ++i) ciclo = pai[ciclo];

    stack<ll> caminho;
    for (ll atual = ciclo; atual = pai[atual]) {
        caminho.push(atual);
        if (atual == ciclo && caminho.size() > 1)
            break;
    }
    while (!caminho.empty()) {
        cout << caminho.top() << ' ';
        caminho.pop();
    }
    cout << '\n';

    return true;
}

```

```
}

```

3.4 SPFA

```
// Shortest Path Faster Algorithm
// Bellman-Ford com fila – relaxa só vértices que
// mudaram
//  $O(V * E)$  pior caso,  $O(E)$  caso médio na prática
//
// Detecta ciclo negativo contando quantas vezes
// cada vértice entra na fila ( $> N$  vezes = ciclo)
//
// Representação: Lista de adjacência (diferente do
// Bellman-Ford)
// Mais eficiente que Bellman-Ford em grafos esparsos
//
// Para caminho máximo: negar todos os pesos
// Para ciclo positivo: negar todos os pesos e detectar
// ciclo negativo

const ll INF = 1e18;
vector<pair<int, ll>> adj[MXN]; // adj[u] = {v, peso}
vector<ll> dist, pai;
vector<int> cnt; // quantas vezes cada vértice entrou
// na fila
vector<bool> inQueue;
int n, m;

// Retorna true se há ciclo negativo alcançável a
// partir de s
bool spfa(int s) {
    dist.assign(n + 1, INF);
    pai.assign(n + 1, -1);
    cnt.assign(n + 1, 0);
    inQueue.assign(n + 1, false);

    dist[s] = 0;
    queue<int> q;
    q.push(s);
    inQueue[s] = true;
    cnt[s] = 1;

    while (!q.empty()) {
        int u = q.front(); q.pop();
        inQueue[u] = false;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pai[v] = u;
                if (!inQueue[v]) {
                    q.push(v);
                    inQueue[v] = true;
                    cnt[v]++;
                    // Vértice entrou na fila mais de N
                    // vezes → ciclo negativo
                    if (cnt[v] > n) return true;
                }
            }
        }
    }

    return false;
}

// Retorna true se existe qualquer ciclo negativo no
// grafo
// (incluindo componentes não alcançáveis por s)
// Inicializa dist=0 para todos: simula supersource
bool acharCicloNegativo() {
    dist.assign(n + 1, 0);
    pai.assign(n + 1, -1);
    cnt.assign(n + 1, 0);
    inQueue.assign(n + 1, false);

    queue<int> q;
    for (int i = 1; i <= n; i++) {
        q.push(i);
        inQueue[i] = true;
        cnt[i] = 1;
    }

    while (!q.empty()) {

```

```
int u = q.front(); q.pop();
inQueue[u] = false;

for (auto [v, w] : adj[u]) {
    if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        pai[v] = u;
        if (!inQueue[v]) {
            q.push(v);
            inQueue[v] = true;
            cnt[v]++;
            if (cnt[v] > n) return true;
        }
    }
}
}

return false;
}

```

3.5 Floyd-Warshall

```
//  $O(V^3)$ ;  $S(V^2)$ 
int n, m;
void floyd_warshall(vector<vector<ll>>& d){
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

// no main:
vector<vector<ll>> d(n+1, vector<ll>(n+1, INF));
for (int i = 1; i <= n; ++i)
    d[i][i] = 0; // dist = 0 para auto-laço

```

3.6 Caminho Euleriano

```
// Algoritmo de Hierholzer
// Para grafos não direcionados:
//  $O(V + E)$ 

vector<vector<pair<int, int>>> adj; // {vizinho,
// id_aresta}
vector<bool> usado;
int n, m;

int temCaminhoEuleriano() {
    int ini = -1, impar = 0;

    for (int i = 1; i <= n; i++) {
        if (!adj[i].empty() && ini == -1) ini = i;
        if (adj[i].size() & 1)
            impar++;
    }

    if (ini == -1) return 2;
    if (impar != 0 && impar != 2) return 0;

    vector<int> ptr(n + 1, 0); // próxima aresta a
    // visitar
    stack<int> st;
    vector<int> caminho;
    st.push(ini);

    while (!st.empty()) {
        int u = st.top();
        // Avança ptr até achar aresta não usada
        while (ptr[u] < (int)adj[u].size() && usado[adj[u][ptr[u]].second])
            ptr[u]++;
        if (ptr[u] < (int)adj[u].size()) {
            auto [v, id] = adj[u][ptr[u]++];
            usado[id] = true;
            st.push(v);
        }
        else {
            caminho.push_back(u);
            st.pop();
        }
    }
}

```

```

for (int i = 0; i < m; i++)
    if (!usado[i]) return 0;

reverse(caminho.begin(), caminho.end());
for (auto i : caminho) cout << i << ' ';
cout << '\n';

return impar == 0 ? 2 : 1;
}

// Para grafos direcionados:
// O(V + E)
vector<vector<int>> adj;
vector<int> in, out;
int n;
int temCaminhoEuleriano() {
    int ini = -1, fim = -1, temCiclo = 1;

    for (int i = 1; i <= n; i++) {
        if (out[i] - in[i] == 1) {
            if (ini != -1) return 0; // Invalido: soh 1
            // no pode ter out > in
            ini = i;
        }
        else if (in[i] - out[i] == 1) {
            if (fim != -1) return 0; // Invalido: soh 1
            // no pode ter in > out
            fim = i;
        }
        else if (in[i] != out[i])
            return 0; // Invalido: |in-out| de um no
            // NAO pode ser > 1
    }

    // Se não achou ini, é circuito
    if (ini == -1) {
        temCiclo = 2;
        for (int i = 1; i <= n; i++) {
            if (out[i] > 0) {
                ini = i;
                break;
            }
        }
    }

    if (ini == -1) return 0;

    // Algoritmo de Hierholzer
    stack<int> st;
    vector<int> caminho;
    st.push(ini);

    while (!st.empty()) {
        int u = st.top();
        if (!adj[u].empty()) {
            int v = adj[u].back();
            adj[u].pop_back();
            st.push(v);
        }
        else {
            caminho.push_back(u);
            st.pop();
        }
    }

    // Verifica se o grafo é conexo
    for (int i = 1; i <= n; i++)
        if (!adj[i].empty()) return 0;

    reverse(caminho.begin(), caminho.end());
    for (auto& i : caminho)
        cout << i << ' ';
    cout << '\n';

    return temCiclo; // 1 = Caminho, 2 = Ciclo
}

```

3.7 Componentes Biconexos (BCC)

3.8 Componentes Fortemente Conexos (SCC)

```

// O(V + E) - Algoritmo de Kosaraju
// Encontra todos os componentes fortemente conexos em
// grafo direcionado

const int MAXN = 1e5+1;
vector<vector<int>> grafo, grafoReverso;
vector<int> ordem; // Ordem topológica
bitset<MAXN> visitado;
int n, m; // vértices, arestas

// ordenação topológica
void dfsOrdem(int u) {
    visitado[u] = true;
    for (auto& v : grafo[u])
        if (!visitado[v])
            dfsOrdem(v);
    ordem.push_back(u);
}

// DFS no grafo reverso para encontrar SCCs
void dfsComponente(int u) {
    visitado[u] = true;
    for (auto& v : grafoReverso[u])
        if (!visitado[v])
            dfsComponente(v);
}

// Retorna vetor com um representante de cada
// componente
vector<int> kosaraju() {
    for (int i = 1; i <= n; ++i)
        if (!visitado[i])
            dfsOrdem(i);
    reverse(ordem.begin(), ordem.end());

    visitado.reset();
    vector<int> componentes;

    for (auto& u : ordem) {
        if (!visitado[u]) {
            dfsComponente(u);
            componentes.push_back(u);
        }
    }

    return componentes;
}

```

3.9 Árvore Geradora Mínima (MST)

```

// Prim
// O((V + E) * LogV)

const int MAXN = 1e5+1;
bitset<MAXN> visitado;

int prim() {
    priority_queue<pair<int,int>> pq; // {peso, no}
    pq.emplace(0,1); // começa no no 1

    int sum = 0;
    while (!pq.empty()) {
        auto [w, u] = pq.top();
        pq.pop();

        if (visitado[u]) continue;

        sum += -w;
        visitado[u] = true;

        for (auto& [v,w] : adj[u]) // adj = {no, peso}
            if (!visitado[v])
                pq.emplace(-w, v);
    }

    return visitado[n]? sum : -1;
}

// Kruskal (usa Union-find)

```

```
// O(E * LogE)
// N precisa guardar grafo, soh arestas
struct Edge {
    int a,b, c;
};
vector<Edge> arestas;
vector<int> p;

int kruskal() {
    for (int i = 1; i <= n; ++i) p[i] = i;
    sort(arestas.begin(), arestas.end(), [](auto a,
        auto b){
            return a.c < b.c;
        });

    int sum = 0;
    for (auto& e : arestas) {
        if (find(e.a) != find(e.b)) {
            join(e.a, e.b);
            sum += e.c;
        }
    }

    return find(1)==find(n)? sum : -1;
}

int find(int x) {
    if (x == p[x]) return x;
    return p[x] = find(p[x]);
}

void join(int x, int y) {
    p[find(x)] = find(y);
}
```

4 Programação Dinâmica

4.1 Edit Distance

```
// Levenshtein Distance
// Problema: menor custo para transformar string s em t
// Operações: inserir (custo 1), remover (custo 1),
// substituir (custo (s[i]!=t[j]))
// DP 2D: O(N*M) tempo, O(N*M) espaço
int editDistance(string& s, string& t) {
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1, 1e9));
    dp[0][0] = 0;
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i) dp[i][j] = min(dp[i][j], dp[i-1][j]
                + 1); // remove
            if (j) dp[i][j] = min(dp[i][j], dp[i][j-1]
                + 1); // insere
            if (i && j) dp[i][j] = min(dp[i][j], dp[i-1][j-1]
                + (s[i-1] != t[j-1])); // substitui
        }
    }
    return dp[n][m];
}

// DP 1D: O(N*M) tempo, O(M) espaço
int editDistance1D(string& s, string& t) {
    int n = s.size(), m = t.size();
    vector<int> prev(m+1), cur(m+1);
    iota(prev.begin(), prev.end(), 0);
    for (int i = 1; i <= n; i++) {
        cur[0] = i;
        for (int j = 1; j <= m; j++) {
            cur[j] = min({prev[j] + 1, cur[j-1] + 1,
                prev[j-1] + (s[i-1] != t[j-1])});
        }
        swap(prev, cur);
    }
    return prev[m];
}
```

4.2 Problema da Mochila

```
// Knapsack 0/1 - cada item usado no máximo 1 vez
// dp[j] = maior valor com capacidade exatamente j
```

```
// Itera j de trás pra frente: evita usar o item duas
vezes
// O(N * W)
void knapsack01(int n, int W, vector<ll>& w, vector<ll>
    & v) {
    vector<ll> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int j = W; j >= w[i]; j--)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    cout << dp[W] << '\n';
}

// Knapsack ilimitado - cada item pode ser usado
infinitas vezes
// Itera j da frente pra trás: permite reusar o mesmo
item
// O(N * W)
void knapsack(int n, int W, vector<ll>& w, vector<ll>&
    v) {
    vector<ll> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int j = w[i]; j <= W; j++)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    cout << dp[W] << '\n';
}

// Knapsack 0/1 - quando W é muito grande, inverte
dimensões
// dp[j] = menor peso para atingir exatamente valor j
// Útil quando V (soma dos valores) << W
// O(N * V)
void knapsack01ByValue(int n, int W, vector<ll>& w,
    vector<ll>& v) {
    ll total = 0;
    for (ll x : v) total += x;
    vector<ll> dp(total + 1, LLONG_MAX / 2);
    dp[0] = 0;
    for (int i = 0; i < n; i++)
        for (int j = total; j >= v[i]; j--)
            dp[j] = min(dp[j], dp[j - v[i]] + w[i]);
    ll ans = 0;
    for (ll j = 0; j <= total; j++)
        if (dp[j] <= W) ans = j;
    cout << ans << '\n';
}
```

4.3 Máscara de Bits

```
// Operações com Bitmask
// Índices 0-based. "bit i" = posição i da direita.
// Uso típico: representar subconjuntos de {0, 1, ...,
N-1}

// --- Operações básicas ---
mask | (1 << i) // liga bit i
mask & ~(1 << i) // desliga bit i
mask ^ (1 << i) // inverte bit i
(mask >> i) & 1 // lê bit i (0 ou 1)
mask & (1 << i) // testa bit i (0 se desligado)

// --- Propriedades do conjunto ---
__builtin_popcount(mask) // quantos bits ligados
__builtin_ctz(mask) // índice do bit menos
significativo ligado
__builtin_clz(mask) // zeros à esquerda (cuidado
: UB se mask==0)
(mask & (mask-1)) == 0 // mask é potência de 2 (
exatamente 1 bit)
mask == 0 // conjunto vazio

// --- Iteração ---

// Todos os subconjuntos de {0..N-1}
for (int mask = 0; mask < (1 << n); mask++) { }

// Todos os bits ligados de mask
for (int tmp = mask; tmp; tmp &= tmp-1) {
    int i = __builtin_ctz(tmp); // índice do bit
}

// Todos os subconjuntos de mask (incluindo vazio)
for (int sub = mask; sub > 0; sub = (sub-1) & mask) { }
```

```
// Atenção: não itera o subconjunto vazio; adicione
// caso sub==0 se necessário

// --- Máscaras úteis ---
(1 << n) - 1 // todos os n bits ligados
(1 << i) // só o bit i ligado

// --- Padrão DP em subconjuntos ---
// dp[mask][v] = valor considerando exatamente os
// vértices em mask,
// terminando/passando em v
// Transição típica:
// int prev = mask ^ (1 << v); // remove v do
// conjunto
// dp[mask][v] = f(dp[prev][u]) para u vizinho de v
// em prev
//
// Exemplo (Hamiltonian Path - O(2^N * N^2)):
// dp[mask][v] += dp[mask ^ (1<<v)][u] * adj[u][v]
// para todo u em (mask ^ (1<<v)) com adj[u][v] > 0
```

5 Matemática

5.1 Fórmulas

PROGRESSÕES E SOMAS

Soma de consecutivos (PA):
 $S = (\text{primeiro} + \text{último}) * n / 2$

Soma de 1 até N:
 $S = N * (N + 1) / 2$

Soma de quadrados de 1 até N:
 $S = N * (N + 1) * (2N + 1) / 6$

Soma de cubos de 1 até N:
 $S = (N * (N + 1) / 2)^2$

Soma de PG:
 $S = a * (r^n - 1) / (r - 1) \quad (r \neq 1)$

Soma de PG infinita ($|r| < 1$):
 $S = a / (1 - r)$

Termo geral da PA:
 $a_n = a_1 + (n - 1) * d$

Termo geral da PG:
 $a_n = a_1 * r^{(n-1)}$

COMBINATÓRIA

Fatorial:
 $n! = n * (n-1) * \dots * 1$
 $0! = 1$

Combinação:
 $C(n, k) = n! / (k! * (n-k)!)$
 $C(n, 0) = C(n, n) = 1$
 $C(n, k) = C(n-1, k-1) + C(n-1, k) \quad (\text{triângulo de Pascal})$

Permutação:
 $P(n, k) = n! / (n-k)!)$

Permutação com repetição:
 $P = n! / (n_1! * n_2! * \dots * n_k!)$

Arranjo:
 $A(n, k) = n! / (n-k)!)$

Número de subconjuntos de N elementos:
 2^N

Princípio da inclusão-exclusão:
 $|A \cup B| = |A| + |B| - |A \cap B|$
 $|A \cup B \cup C| = |A| + |B| + |C| - |nAB| - |nAC| - |nBC| + |nABC|$

Números de Catalan:

$C_n = C(2n, n) / (n + 1)$
 $C_0=1, C_1=1, C_2=2, C_3=5, C_4=14, \dots$
 Uso: árvores binárias, parênteses válidos, triangulações

Coefficiente binomial – identidades úteis:

$C(n, k) = C(n, n-k)$
 $C(n, k)$ para $k=0..n = 2^n$
 $C(n, k)^2$ para $k=0..n = C(2n, n)$

TEORIA DOS NÚMEROS

Divisibilidade – número de divisores:

Se $n = p_1^{a_1} * p_2^{a_2} * \dots * p_k^{a_k}$
 $d(n) = (a_1+1) * (a_2+1) * \dots * (a_k+1)$

Soma dos divisores: σ

$(n) = \prod (p_i^{a_i+1} - 1) / (p_i - 1)$

Função de Euler (totiente): ϕ

$(n) = n * \prod (1 - 1/p)$ para p primo divisor de n
 $(1) = 1$
 Se p primo: $\phi(p) = p - 1$
 Se p primo: $\phi(p^k) = p^k - p^{(k-1)}$

Pequeno Teorema de Fermat:

$a^p \equiv a \pmod{p}$ para p primo
 $a^{(p-1)} \equiv 1 \pmod{p}$ para p primo e $\text{mdc}(a,p)=1$

Teorema de Euler:

$a^{\phi(n)} \equiv 1 \pmod{n}$ para $\text{mdc}(a,n)=1$

Inverso modular (p primo):

$a^{(-1)} \equiv a^{(p-2)} \pmod{p}$

Teorema Chinês **do** Resto (CRT):

Sistema $x \equiv a_i \pmod{m_i}$ com m_i coprimos dois a dois:
 $x = \sum a_i * M_i * y_i \pmod{M}$
 onde $M = \prod m_i$, $M_i = M/m_i$, $y_i = M_i^{(-1)} \pmod{m_i}$

MDC / MMC:

$\text{mdc}(a, b) = \text{mdc}(b, a \bmod b)$
 $\text{mmc}(a, b) = a * b / \text{mdc}(a, b)$
 $\text{mdc}(a, 0) = a$

Identidade de Bézout:

$\text{mdc}(a, b) = a*x + b*y \quad (x, y \text{ inteiros})$

GEOMETRIA

Distância euclidiana:

$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$

Distância Manhattan:

$d = |x_2-x_1| + |y_2-y_1|$

Área **do** triângulo (coordenadas):

$A = |x_1*(y_2-y_3) + x_2*(y_3-y_1) + x_3*(y_1-y_2)| / 2$

Produto vetorial 2D:

$(a \times b) = a.x*b.y - a.y*b.x$
 > 0 : b está à esquerda de a (anti-horário)
 < 0 : b está à direita de a (horário)
 $= 0$: colineares

Produto escalar:

$a \cdot b = a.x*b.x + a.y*b.y = |a||b|\cos\theta()$

Área **do** polígono (Shoelace):

$A = \sum |x_i * y_{i+1} - x_{i+1} * y_i| / 2$

Teorema de Pick (polígono em grade inteira):

$A = I + B/2 - 1$
 I = pontos inteiros internos
 B = pontos inteiros na borda

A = área (Shoelace)

Círculo:

$$\begin{aligned} \text{Área} &= \pi * r^2 \\ \text{Perímetro} &= 2 * \pi * r \end{aligned}$$

Setor circular:

$$\text{Área} = \theta * r^2 / 2 \theta \quad (\text{em radianos})$$

Fórmula de Herão (triângulo, lados a,b,c):

$$\begin{aligned} s &= (a + b + c) / 2 \\ A &= \sqrt{s * (s-a) * (s-b) * (s-c)} \end{aligned}$$

TEORIA DOS GRAFOS

Handshaking lemma: $\sum$

$$\text{grau}(v) = 2 * |E|$$

Árvore com N vértices:

$$|E| = N - 1$$

Grafo euleriano (não direcionado):

Circuito: todos os graus pares + conexo
Caminho: exatamente 2 vértices de grau ímpar + conexo

Grafo euleriano (direcionado):

Circuito: $\text{in}[v] = \text{out}[v]$ para todo v + conexo
Caminho: um v com $\text{out}-\text{in}=1$ (início), um com $\text{in}-\text{out}=1$ (fim), resto iguais

Fórmula de Euler (grafo planar):

$$V - E + F = 2 \quad (V \text{ vértices, } E \text{ arestas, } F \text{ faces incluindo externa})$$

PROBABILIDADE

Probabilidade condicional:

$$P(A|B) = P(A \cap B) / P(B)$$

Teorema de Bayes:

$$P(A|B) = P(B|A) * P(A) / P(B)$$

Valor esperado:

$$\begin{aligned} E[X] &= \sum x_i * P(x_i) \\ E[aX + b] &= a * E[X] + b \\ E[X + Y] &= E[X] + E[Y] \end{aligned}$$

Variância:

$$\text{Var}(X) = E[X^2] - E[X]^2$$

EXPONENCIAÇÃO E LOGARITMOS

Exponenciação rápida:

a^n em $O(\log n)$ por repeated squaring

Identidades de log:

$$\begin{aligned} \log(a*b) &= \log(a) + \log(b) \\ \log(a/b) &= \log(a) - \log(b) \\ \log(a^b) &= b * \log(a) \\ \log_b(a) &= \log(a) / \log(b) \end{aligned}$$

Número de dígitos de N na base B:

$$d = \text{floor}(\log_B(N)) + 1$$

BITMASK

Número de subconjuntos de N elementos:

$$2^N$$

Soma de todos os tamanhos de subconjuntos:

$$N * 2^{(N-1)}$$

STRINGS

Número de substrings de string de tamanho N:
 $N * (N + 1) / 2$

MISCELÂNEA

Princípio das casas de pombo (Pigeonhole):

$N+1$ objetos em N caixas $\rightarrow$ alguma caixa tem ≥ 2 objetos

Soma dos primeiros N ímpares:

$$1 + 3 + 5 + \dots + (2N-1) = N^2$$

Potências de 2:

$$\begin{aligned} 2^{10} &= 1.024 & (\sim 10^3) \\ 2^{20} &= 1.048.576 & (\sim 10^6) \\ 2^{30} &= 1.073.741.824 & (\sim 10^9) \end{aligned}$$

Limite **int** (32 bits):

$$\begin{aligned} \text{INT_MAX} &= 2^{31} - 1 = 2.147.483.647 & (\sim 2 \cdot 10^9) \\ \text{INT_MIN} &= -2^{31} = -2.147.483.648 \end{aligned}$$

Limite **long long** (64 bits):

$$\begin{aligned} \text{LLONG_MAX} &= 2^{63} - 1 & (\sim 9.2 \cdot 10^{18}) \\ \text{LLONG_MIN} &= -2^{63} & (\sim -9.2 \cdot 10^{18}) \end{aligned}$$

5.2 MDC & MMC

```
// O(logN); n = min(a,b)
// existe a função gcd()
11 mdc(11 a, 11 b) {
    if (b == 0) return a;
    return mdc(b, a%b);
}
```

```
// existe a função lcm()
11 mmc(11 a, 11 b) {
    return (a*b)/mdc(a,b);
}
```

5.3 Exponenciação Binária

```
// a^b O(logN)
11 bin_pow(11 a, 11 b, const 11 MOD = 1e9 + 7) {
    a %= MOD;
    11 res = 1;

    while (b) {
        if (b % 2) res = res * a % MOD;
        a = a * a % MOD;
        b /= 2;
    }
    return res;
}

11 inverso(11 a, 11 MOD) {
    bin_pow(a, MOD-2); // MOD deve ser primo
}
```

5.4 Primos

mashu lucky prime : 91145149
1097774749, 1076767633, 100102021, 999997771
1001010013, 1000512343, 987654361, 999991231
999888733, 98789101, 987777733, 999991921, 1010101333

5.5 Crivo de Eratóstenes

```
// O(N * Log(LogN))
vector <bool> eh_primo(TAM, true);
void crivo(int n) {
    eh_primo[0] = eh_primo[1] = false;
    for (int p = 2; p * p <= n; p++) {
        if (eh_primo[p]) {
            for (int i = p * p; i <= n; i += p)
                eh_primo[i] = false;
        }
    }
}
```



```

    }
  }
}

```

5.6 Fatoração Prima

```

// Retorna mapa {primo -> expoente}
// Exemplo: 12 = 2^2 * 3^1 -> {{2,2},{3,1}}

// --- Fatoração de um único número ---
// O(sqrt(N))
map<int,int> fatorar(int n) {
  map<int,int> fat;
  for (int p = 2; p * p <= n; p++) {
    while (n % p == 0) {
      fat[p]++;
      n /= p;
    }
  }
  // Fator primo restante > sqrt(N)
  if (n > 1) fat[n]++;
  return fat;
}

// --- Menor fator primo (SPF) ---
// Pré-computa spf[i] = menor fator primo de i
// Pré-processamento: O(N * Log(LogN))
// Fatoração de qualquer n <= N: O(LogN)
int spf[MXN];
void buildSPF(int n) {
  for (int i = 1; i <= n; i++) spf[i] = i;
  for (int p = 2; p * p <= n; p++) {
    if (spf[p] == p) { // p é primo
      for (int j = p * p; j <= n; j += p)
        if (spf[j] == j) spf[j] = p; // marca menor fator
    }
  }
}

map<int,int> fatorarSPF(int n) {
  map<int,int> fat;
  while (n > 1) {
    fat[spf[n]]++;
    n /= spf[n];
  }
  return fat;
}

```

5.7 Função Totiente de Euler (ϕ)

```

//  $\phi(n)$  = quantidade de inteiros em [1, n] coprimos com n
// Propriedades:
//  $\phi(1) = 1$ 
// p primo  $\rightarrow \phi(p) = p - 1$ 
// p primo  $\rightarrow \phi(p^k) = p^k - p^{k-1}$ 
//  $\text{mdc}(a,b)=1 \rightarrow \phi(a*b) = \phi(a) * \phi(b)$  (multiplicativa)
//
// Fórmula:  $\phi(n) = n * \prod (1 - 1/p)$  para todo primo p | n
//          =  $n * \prod (p-1)/p$ 
// Exemplo:  $\phi(12) = 12 * (1-1/2) * (1-1/3) = 12 * 1/2 * 2/3 = 4$ 
//
// Uso com exponenciação modular:
//  $a^b \bmod c = a^{(b \% \phi(c) + \phi(c))} \bmod c$  se  $b \geq \phi(c)$ 
//          =  $a^b \bmod c$  se  $b < \phi(c)$ 
//
// --- Totiente de um único número ---
// O(sqrt(N))
int phi(int n) {
  int res = n;
  // Fatoramos n e para cada primo p encontrado
  // aplicamos res = res * (p-1)/p
  // (equivalente a res -= res/p, evitando divisão com perda)
  for (int p = 2; p * p <= n; p++) {

```

```

    if (n % p == 0) {
      // p é fator primo de n
      res = res / p * (p - 1); // res *= (1 - 1/p)
      while (n % p == 0) n /= p; // remove todas as ocorrências de p
    }
  }
  // Se ainda sobrou um fator > 1, ele é primo
  if (n > 1) res = res / n * (n - 1);
  return res;
}

// --- Tabela de totientes para [1, n] ---
// O(N * Log(LogN)) - igual ao crivo de Eratóstenes
// Ideia: para cada primo p, multiplica todos os múltiplos por (p-1)/p
int phi_table[MXN];
void phitable(int n) {
  // Inicializa  $\phi(i) = i$  para todo i (valor antes de aplicar os fatores)
  for (int i = 1; i <= n; i++) phi_table[i] = i;

  for (int p = 2; p <= n; p++) {
    // Se  $\phi(p)$  ainda é p, então p é primo (não foi modificado por nenhum fator)
    if (phi_table[p] == p) {
      // Aplica o fator (p-1)/p em todos os múltiplos de p
      for (int j = p; j <= n; j += p)
        phi_table[j] = phi_table[j] / p * (p - 1);
    }
  }
}

```

6 Strings

6.1 Arranjo de Sufixos (SA)

6.2 KMP (Knuth-Morris-Pratt)

```

// p[i] = comprimento do maior prefixo próprio do padrão s que também é sufixo
// do texto t[0..i] Prefixo/Sufixo próprio é um prefixo /sufixo que não é a
// string inteira Usos:
// - Busca de padrão único (igual Z, mas mais natural para busca online)
// - Busca online/streaming: processa t caractere a caractere sem ter t
// completo
// - Menor período de string: n - lps[n-1] é o período mínimo
// - Autômato de string: transformar lps em DFA para múltiplas consultas no mesmo padrão
//
// O(m), m = s.size()
vector<int> pi(string s) {
  vector<int> p(s.size());
  for (int i = 1, j = 0; i < s.size(); ++i) {
    while (j > 0 && s[j] != s[i]) j = p[j - 1];
    if (s[j] == s[i]) j++;
    p[i] = j;
  }
  return p;
}

// O(n + m) KMP: busca padrao s em texto t, retorna índices de cada ocorrência
// (0-based)
vector<int> kmp(string &t, string &s) {
  vector<int> p = pi(s + '$'), match;
  for (int i = 0, j = 0; i < t.size(); ++i) {
    while (j > 0 && s[j] != t[i]) j = p[j - 1];
    if (s[j] == t[i]) j++;
    if (j == s.size()) match.push_back(i - j + 1);
  }
  return match;
}

```

6.3 Rabin-Karp

```
// O(N + M) médio, O(N * M) pior caso
// Busca de padrão p em texto t usando hashing de
// janela deslizante
// Usos:
// - Busca de múltiplos padrões simultaneamente (coloca
// todos num set de hashes)
// - Detectar strings repetidas / substrings duplicadas
// (+busca binária)

const ll MOD = 1e9 + 7, BASE = 31;
vector<int> rabin_karp(string t, string p) {
    int n = t.size(), m = p.size();
    vector<int> ocorrencias;

    // Pré-computa potências de BASE
    vector<ll> pw(max(n, m) + 1);
    pw[0] = 1;
    for (int i = 1; i <= max(n, m); i++) pw[i] = pw[i - 1] * BASE % MOD;

    // Hash do padrão
    ll hp = 0;
    for (int i = 0; i < m; i++) hp = (hp + (p[i] - 'a' + 1) * pw[i]) % MOD;

    // Hash prefixo do texto: h[i] = hash(t[0..i-1])
    vector<ll> h(n + 1, 0);
    for (int i = 0; i < n; i++) h[i + 1] = (h[i] + (t[i] - 'a' + 1) * pw[i]) % MOD;

    // Hash de t[l..r-1] = (h[r] - h[l]) / pw[l]
    // = (h[r] - h[l]) * inv(pw[l])
    // Evita inversão modular: compara (h[r] - h[l])
    // com hp * pw[l]
    for (int i = 0; i + m <= n; i++) {
        ll ht = (h[i + m] - h[i] + MOD) % MOD;
        if (ht == hp * pw[i] % MOD) ocorrencias.push_back(i);
    }

    return ocorrencias; // índices (0-based) onde p
    ocorre em t
}
```

6.4 Trie

```
// Armazena strings para buscas eficientes por prefixo
// O(N) por inserção e busca, onde N = tamanho da
// string
// Usos comuns em CP:
// - Contar strings com determinado prefixo
// - Verificar se string/prefixo existe
// - XOR máximo com Trie binária (ver abaixo)
// Como usar:
// Trie t;
// t.insert("hello");
// t.search("hello") → true (string completa existe)
// t.search("hell") → false (prefixo existe mas
// string não foi inserida)
// t.startsWith("hell") → true (prefixo existe)
// t.count("hel") → quantas strings inseridas
// começam com "hel"

struct Trie {
    struct Node {
        int filho[26]; // índice dos filhos (a=0, b=1,
        ..., z=25)
        int fim; // quantas strings terminam
        aqui
        int passa; // quantas strings passam por
        este nó
        Node() : fim(0), passa(0) { fill(filho, filho + 26, -1); }
    };
    vector<Node> nos;
```

```
Trie() { nos.emplace_back(); } // nó raiz = índice 0

void insert(const string& s) {
    int cur = 0;
    for (char c : s) {
        int ch = c - 'a';
        if (nos[cur].filho[ch] == -1) {
            nos[cur].filho[ch] = nos.size();
            nos.emplace_back();
        }
        cur = nos[cur].filho[ch];
        nos[cur].passa++;
    }
    nos[cur].fim++;
}

// Retorna true se a string completa foi inserida
bool search(const string& s) {
    int cur = 0;
    for (char c : s) {
        int ch = c - 'a';
        if (nos[cur].filho[ch] == -1) return false;
        cur = nos[cur].filho[ch];
    }
    return nos[cur].fim > 0;
}

// Retorna true se alguma string inserida começa
// com s
bool startsWith(const string& s) {
    int cur = 0;
    for (char c : s) {
        int ch = c - 'a';
        if (nos[cur].filho[ch] == -1) return false;
        cur = nos[cur].filho[ch];
    }
    return true;
}

// Quantas strings inseridas começam com o prefixo
// s
int count(const string& s) {
    int cur = 0;
    for (char c : s) {
        int ch = c - 'a';
        if (nos[cur].filho[ch] == -1) return 0;
        cur = nos[cur].filho[ch];
    }
    return nos[cur].passa;
}
};
```

6.5 Função Z

```
// O(n)
// z[i] = comprimento do maior prefixo de s que também
// é prefixo de s[i..]
// Usos:
// - Busca de padrão: z(p + "#" + t), ocorrências
// onde z[i] == |p|
// - Menor período de string: menor k tal que k | n
// e z[k] == n - k
// - Contar prefixos distintos que ocorrem em s:
// acumular z[i] com cuidado
// - Comparar sufixos rapidamente sem suffix array
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);

        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;

        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
}
```

```
    return z;
}
```

6.6 Manacher

```
// Encontra o maior palíndromo centrado em cada posição
// O(N) - linear, sem expansão quadrática
//
// Ideia central: reaproveita palíndromos já calculados
// usando a simetria do espelho em torno do centro
// atual
//
// String transformada: intercala '#' entre caracteres
// "abc" → "@#a#b#c#$"
// - '@' e '$' são sentinelas (evitam checagem de
//   bounds)
// - '#' entre caracteres unifica palíndromos pares e
//   ímpares
// - p[i] = raio do palíndromo centrado em i na
//   string transformada
// - palíndromo real de tamanho p[i] (par ou ímpar)
//
// Exemplo: "abacaba"
// t = "@#a#b#a#c#a#b#a#$"
// p = 0 1 0 3 0 1 0 7 0 1 0 3 0 1 0
//
// Como usar:
// 1. Chame manacher(s) para pré-computar p[]
// 2. Use getLongest(i, true) → raio do maior
//   palíndromo ímpar centrado em i
//   Use getLongest(i, false) → raio do maior
//   palíndromo par centrado entre i e i+1
// 3. O tamanho real do palíndromo é igual ao raio
//   retornado
//   ímpar: tamanho = 2*raio + 1, começa em i -
//   raio
//   par: tamanho = 2*raio, começa em i -
//   raio + 1
//
// Exemplo de uso (maior palíndromo):
// manacher(s);
// for (int i = 0; i < s.size(); i++) {
//     int r = getLongest(i, true); // ímpar
//     // palíndromo: s.substr(i - r, 2*r + 1)
//     int r = getLongest(i, false); // par
//     // palíndromo: s.substr(i - r + 1, 2*r)
// }
```

```
vector<int> p;
```

```
void manacher(const string& s) {
    string t = "@";
    for (char c : s) t += "#" + string(1, c);
    t += "$";

    int n = t.size();
    p.assign(n, 0);

    int c = 0, r = 0;
    for (int i = 1; i < n - 1; i++) {
        int mirror = 2 * c - i;
        if (i < r) p[i] = min(r - i, p[mirror]);
        while (t[i + p[i] + 1] == t[i - p[i] - 1]) p[i]++;
        if (i + p[i] > r) { c = i; r = i + p[i]; }
    }
}
```

```
// ímpar: centro real em t[2*i + 2] (o próprio
// caractere)
// par: centro real em t[2*i + 3] (o '#' entre i e
// i+1)
int getLongest(int i, bool isOdd) {
    return p[2 * i + 2 + !isOdd];
}
```

7 Árvores

7.1 LCA (Menor Ancestral Comum)

7.2 Hash de árvore

7.3 Decomposição Pesada-Leve (HLD)

8 Geometria

9 Algoritmos de Ordenação

9.1 Insertion Sort

```
// O(N^2)
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

9.2 Merge Sort

```
// O(N * LogN)
void merge(vector<int>& vet, int left, int mid, int
right){
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        if (vet[i] <= vet[j])
            temp[k++] = vet[i++];
        else
            temp[k++] = vet[j++];
    }
    while (i <= mid)
        temp[k++] = vet[i++];
    while (j <= right)
        temp[k++] = vet[j++];
    for (i = left, k = 0; i <= right; i++, k++)
        vet[i] = temp[k];
}

// Chamada => merge_sort(vet, 0, vet.size() - 1);
void merge_sort(vector<int>& vet, int left, int right)
{
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(vet, left, mid);
        merge_sort(vet, mid + 1, right);
        merge(vet, left, mid, right);
    }
}
```

9.3 Quicksort

```
// O(N * LogN)
int partition(vector<int>& vet, int left, int right) {
    int mid = left + (right - left) / 2;
    int pivot = vet[mid];
    swap(vet[mid], vet[right]);
    int i = left - 1;
    for (int j = left; j < right; j++) {
        if (vet[j] <= pivot) {
            i++;
            swap(vet[i], vet[j]);
        }
    }
    swap(vet[i + 1], vet[right]);
    return i + 1;
}

// Chamada => quick_sort(v, 0, v.size() - 1);
void quick_sort(vector<int>& vet, int left, int right)
{
    if (left < right) {
        int pivot = partition(vet, left, right);
        quick_sort(vet, left, pivot - 1);
        quick_sort(vet, pivot + 1, right);
    }
}
```

9.4 Counting Sort

```
// O(N + K), K = max_val
void counting_sort(vector<int>& vet, int max_val) {
    vector<int> count(max_val + 1, 0);
    for (int x : vet) count[x]++;
    int idx = 0;
    for (int i = 0; i <= max_val; i++)
        while (count[i]--) vet[idx++] = i;
}
```

10 Problemas Especiais

Papel milimetrado para rascunho



